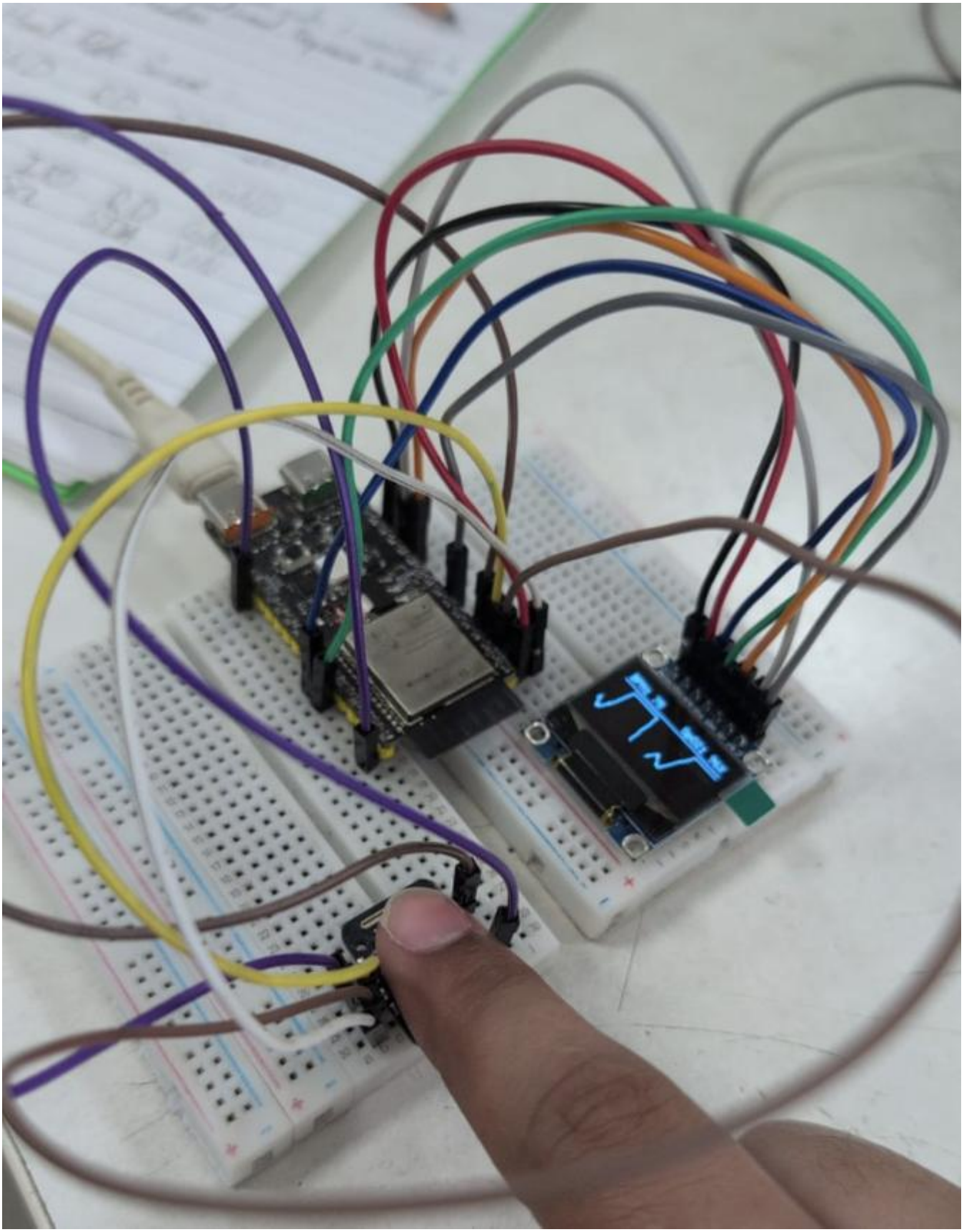


Smart Pulse Oximeter Playbook

I. Challenge

Develop an embedded biometric acquisition terminal using Photoplethysmography (PPG) that isolates high-frequency micro-volumetric blood changes. The system must cleanly separate unshared high-speed hardware bus lines, perform dynamic DC-bias filtering, compute heart rate (BPM) and blood oxygen saturation (SpO_2) over deterministic windowing blocks, and stream real-time physiological waveforms to both an SPI-driven OLED display and diagnostic serial plotters.

II. Guide

A. Concept

i. Underlying Engineering Principles & Reference Links

1. Differential Light Absorption in Hemoglobin: Arterial blood volume pulses during cardiac cycles. Deoxygenated Hemoglobin (Hb) absorbs more Red light (660 nm), whereas Oxygenated Hemoglobin (HbO_2) absorbs more Infrared light (940 nm).
    - ***Learn More:*** NCBI – Principles of Pulse Oximetry
2. **Ratio of Ratios (SpO_2) Calculation:** The optical signal is split into a large static component (DC) from resting tissue and a dynamic component (AC) from pulsating arterial blood. The normalized absorption ratio (R) maps directly to systemic oxygenation:

$$R = \frac{\left(\frac{\mathbf{AC_{Red}}}{\mathbf{DC_{Red}}}\right)}{\left(\frac{\mathbf{AC_{IR}}}{\mathbf{DC_{IR}}}\right)}, \quad \mathbf{SpO_2} = 104 - 17 \cdot R$$

    - ***Learn More:*** Pulse Oximetry Understanding
3. **Exponential Moving Average (EMA) DC Filtering:** Raw photodiode data rides on a massive baseline voltage. To extract the micro-AC waveform for graph rendering, an exponential filter continuously tracks and subtracts the rolling DC component:

$$\mathbf{DC}_t = (\beta \cdot \mathbf{DC}_{t-1}) + \left((1 - \beta) \cdot \mathbf{IR_{Sample}}\right), \quad \mathbf{AC_{Display}} = \mathbf{IR_{Sample}} - \mathbf{DC}_t$$

    - ***Learn More:*** DSP Related – Digital Filters for DC Offset Removal
4. **Deterministic Block Processing Window:** Evaluating metrics sample-by-sample introduces phase jitter, micro-stutters, and computational drift. Collecting 100-sample array buffers (1.0-second windows at 100 Hz) enables global vector calculations without stalling real-time signal acquisition.
    - ***Learn More:*** Block-Based Signal Processing
5. **Bus Isolation Across Mixed Peripherals:** To prevent data corruption on high-speed hardware lines, write-only SPI display lines and bidirectional I^2C sensor buses share system power domains while maintaining strictly isolated controller pins.

- ***Learn More:*** [Communication Protocols]

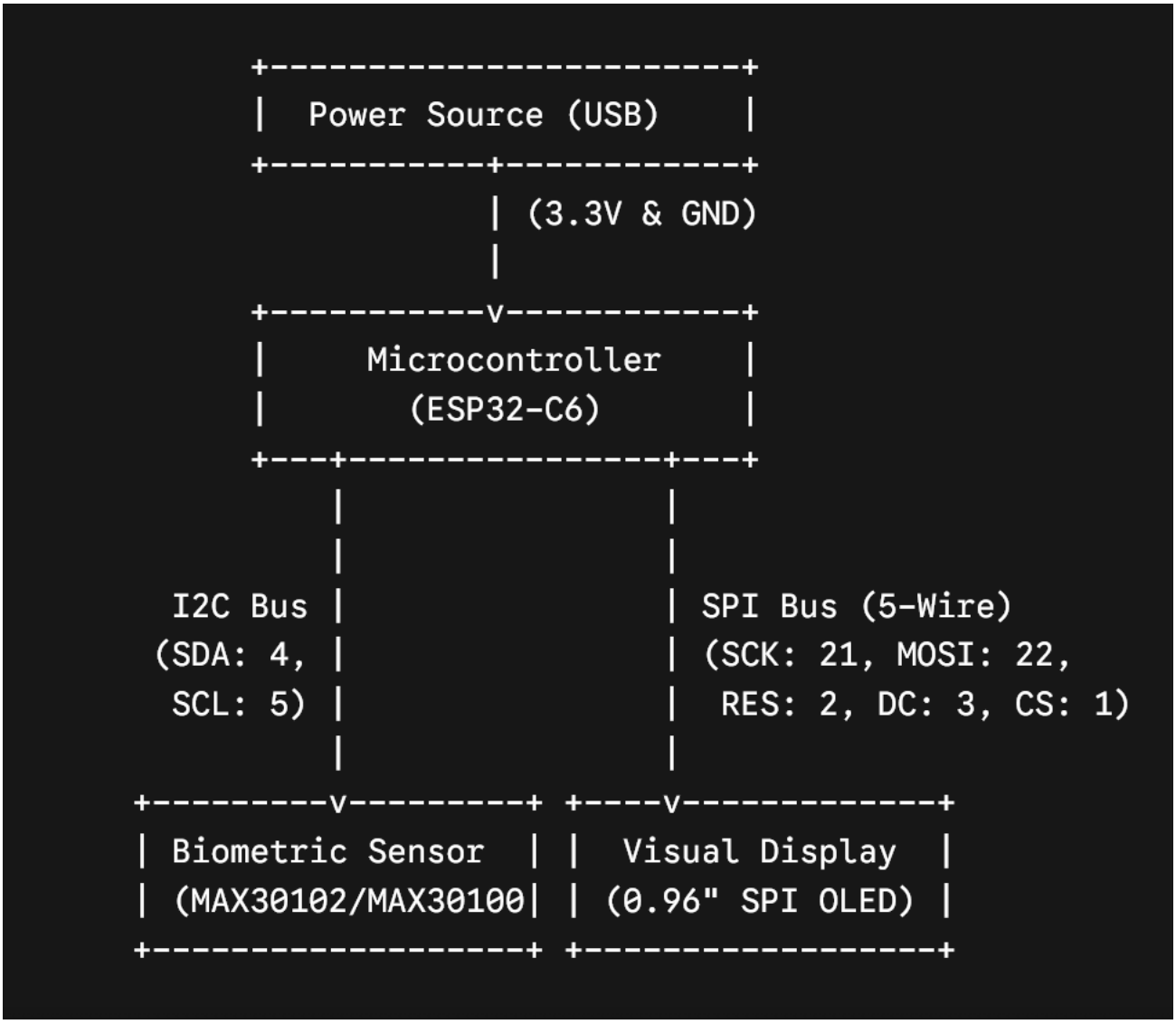
ii. Real-World Example

Medical-grade ICU vital sign monitors, commercial fitness wearables (such as Apple Watch, Garmin, or Fitbit), and emergency pulse oximeter clip units. These systems use optical LEDs and photodiodes to continuously monitor heart rate variability and blood oxygen saturation in real time.

B. Components

- **Microcontroller:** WeAct Studio ESP32-C6 (32-bit RISC-V SoC)
- **Biometric Sensor:** MAX30102 High-Sensitivity Pulse Oximeter & Heart-Rate Breakout
- **Display Peripheral:** 0.96-inch SPI OLED Display (SSD1306 driver)
- **Interconnect Base:** Solderless breadboard and jumper wires

Block Diagram



C. Connections

Sensor/Display Pin	ESP32-C6 Pin	Signal Type	Functional Description
OLED GND / MAX30102 GND	GND	Power	System Common Ground Plane
OLED VDD / MAX30102 VIN	3.3V or 5V	Power	3.3V Logic / 5V LED Headroom Rail
OLED SCK	GPIO 21	Output (FSPI)	Hardware SPI Clock Master Line
OLED SDA	GPIO 22	Output (FSPI)	Hardware SPI Data Line (MOSI)
OLED RES	GPIO 2	Output	Discrete Hardware Reset Flag
OLED DC	GPIO 3	Output	Data / Command Selection ($0 =$ Cmd, $1 =$ Data)
OLED CS	GPIO 1	Output (FSPI)	Active-Low Peripheral Chip Select
MAX30102 SCL	GPIO 5	In/Out (I^2C)	Hardware I^2C Clock Master Line (400 kHz)
MAX30102 SDA	GPIO 4	In/Out (I^2C)	Hardware I^2C Serial Data Line

⚠️ **Hardware Note:** The IRD and RD pins broken out on this MAX30102 variant are left floating. They represent low-side open cathodes of internal LEDs and are managed automatically via internal current-sink transistors in software.

D. Code

```
C++
#include <Arduino.h>
#include <Wire.h>
#include <SPI.h>
#include <U8g2lib.h>
#include "MAX30105.h"

#define SPI_OLED_SCK  21
#define SPI_OLED_SDA  22
#define SPI_OLED_RES  2
#define SPI_OLED_DC   3
#define SPI_OLED_CS   1

#define I2C_SDA       4
#define I2C_SCL       5

U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI oled(U8G2_R0, SPI_OLED_CS, SPI_OLED_DC, SPI_OLED_RES);
MAX30105 particleSensor;

const int SCREEN_WIDTH = 128;
int waveBuffer[SCREEN_WIDTH];

const int BUFFER_SIZE = 100;
uint32_t redBuffer[BUFFER_SIZE];
uint32_t irBuffer[BUFFER_SIZE];
int bufferIndex = 0;

float dcIR = 0;
const float beta = 0.95f;

float finalBpm = 0.0f;
float finalSpO2 = 0.0f;
bool fingerPresent = false;

void setup() {
  Serial.begin(115200);

  Wire.begin(I2C_SDA, I2C_SCL, 400000U);
  SPI.begin(SPI_OLED_SCK, -1, SPI_OLED_SDA, SPI_OLED_CS);
  oled.begin();

  if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
    while (1); // Trap configuration lockup if I2C fails
  }

  // 100Hz dual-mode initialization
  particleSensor.setup(50, 1, 2, 100, 411, 4096);

  for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32;
}

void loop() {
  while (particleSensor.check() > 0) {
    uint32_t redSample = particleSensor.getFIFORed();
    uint32_t irSample = particleSensor.getFIFOIR();

    // 1. Hardware Presence Gate
    if (irSample < 25000) {
      fingerPresent = false;
```

```
      finalBpm = 0.0f;
      finalSpO2 = 0.0f;
      bufferIndex = 0;
      for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32;
      particleSensor.nextSample();
      break;
    }
    fingerPresent = true;

    // 2. Continuous Visual Waveform Filter for OLED Graph
    dcIR = (beta * dcIR) + ((1.0f - beta) * irSample);
    float acDisplay = irSample - dcIR;

    for (int i = 0; i < SCREEN_WIDTH - 1; i++) {
      waveBuffer[i] = waveBuffer[i + 1];
    }
    int yPixel = 42 - (int)(acDisplay * 0.05f);
    waveBuffer[SCREEN_WIDTH - 1] = constrain(yPixel, 18, 54);

    // 3. Populate Array Block
    redBuffer[bufferIndex] = redSample;
    irBuffer[bufferIndex] = irSample;
    bufferIndex++;

    // 4. Vector Analytics Window Execution (Once per 100 samples)
    if (bufferIndex >= BUFFER_SIZE) {
      uint32_t minRed = redBuffer[0], maxRed = redBuffer[0];
      uint32_t minIR  = irBuffer[0],  maxIR  = irBuffer[0];
      uint64_t sumRed = 0, sumIR = 0;

      for (int i = 0; i < BUFFER_SIZE; i++) {
        if (redBuffer[i] < minRed) minRed = redBuffer[i];
        if (redBuffer[i] > maxRed) maxRed = redBuffer[i];
        if (irBuffer[i]  < minIR)  minIR  = irBuffer[i];
        if (irBuffer[i]  > maxIR)  maxIR  = irBuffer[i];
        sumRed += redBuffer[i];
        sumIR  += irBuffer[i];
      }

      float dcR_calc = (float)sumRed / BUFFER_SIZE;
      float dcI_calc = (float)sumIR / BUFFER_SIZE;
      float acR_calc = (float)(maxRed - minRed);
      float acI_calc = (float)(maxIR - minIR);

      // 5. SpO2 Computational Node
      if (acI_calc > 20 && acR_calc > 20) {
        float R = (acR_calc / dcR_calc) / (acI_calc / dcI_calc);
        float spo2Est = 104.0f - (17.0f * R);
        if (spo2Est >= 90.0f && spo2Est <= 100.0f) {
          finalSpO2 = spo2Est;
        } else if (spo2Est < 90.0f && spo2Est >= 75.0f) {
          finalSpO2 = (0.5f * finalSpO2) + (0.5f * spo2Est);
        }
      }

      // 6. Midpoint Inflection Pitch Timing (BPM calculation)
      int lastPeakIdx = -1;
      int totalIntervals = 0;
      int intervalSum = 0;
      float localThresh = dcI_calc;

```

```
      for (int i = 1; i < BUFFER_SIZE - 1; i++) {
        if (irBuffer[i] > localThresh && irBuffer[i-1] <= localThresh) {
          if (lastPeakIdx != -1) {
            intervalSum += (i - lastPeakIdx);
            totalIntervals++;
          }
          lastPeakIdx = i;
        }
      }

      if (totalIntervals > 0) {
        float avgIntervalSamples = (float)intervalSum / totalIntervals;
        float calculatedBpm = (60.0f * 100.0f) / avgIntervalSamples;
        if (calculatedBpm >= 50.0f && calculatedBpm <= 160.0f) {
          finalBpm = calculatedBpm;
        }
      }
      bufferIndex = 0;
    }
    particleSensor.nextSample();
  }

  // 7. Render & Screen UI Generation Path
  oled.clearBuffer();

  if (fingerPresent && finalBpm > 10.0f) {
    for (int x = 1; x < SCREEN_WIDTH; x++) {
      oled.drawLine(x - 1, waveBuffer[x - 1], x, waveBuffer[x]);
    }
    oled.setFont(u8g2_font_6x10_tf);
    oled.setCursor(2, 11);  oled.print("BPM: "); oled.print((int)finalBpm);
    oled.setCursor(72, 11); oled.print("SpO2: "); oled.print((int)finalSpO2);
oled.print("%");
  } else {
    // Standardized typography layout to guarantee 128px bounding limits
    oled.setFont(u8g2_font_5x7_tf);
    oled.setCursor(34, 32); oled.print("PLACE FINGER");
    oled.setCursor(36, 44); oled.print("ON THE LENS");
  }

  oled.drawHLine(0, 14, 128);
  oled.drawHLine(0, 56, 128);
  oled.sendBuffer();
}
```

Student Tip: Open the Serial Monitor (Ctrl+Shift+M) at 115200 baud to monitor real-time raw values and vector computations.

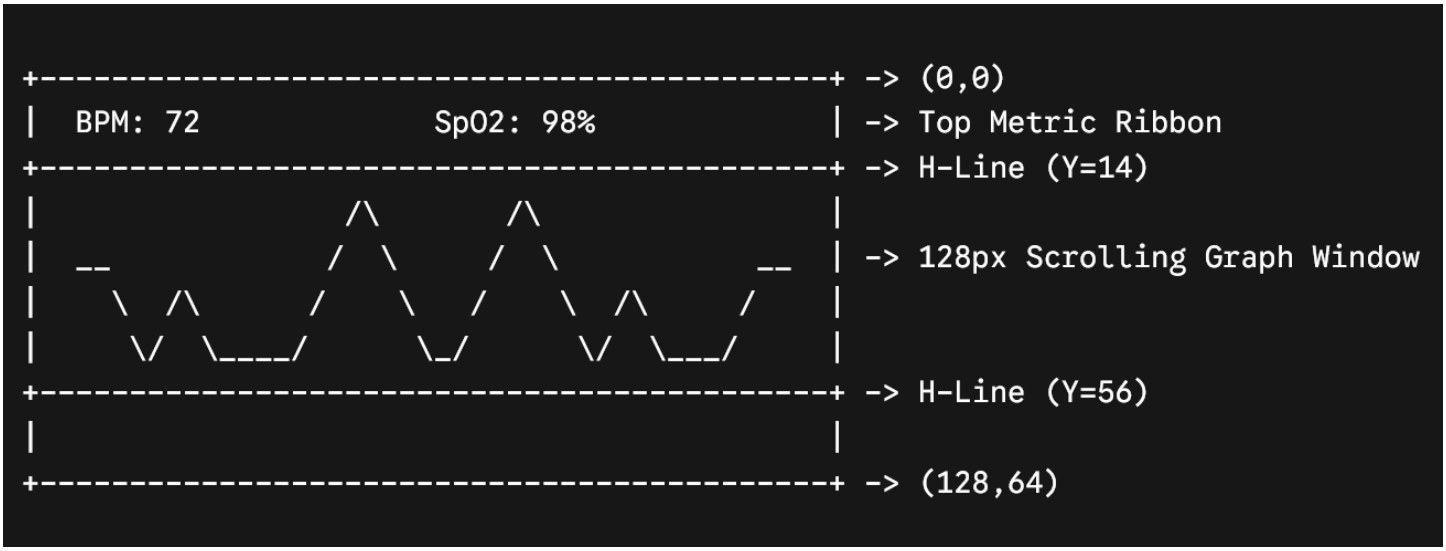
To visually evaluate signal quality and watch the real-time arterial pulse wave, switch to the Serial Plotter (Ctrl+Shift+L). The output telemetry formats IR_Raw, Filtered_AC, BPM, and SpO2 into synchronized multi-colored curves. Watching Filtered_AC helps demonstrate how motion artifacts or excessive contact pressure flatten the cardiac pulse wave.

Operational Guidelines & UI Validation

1. Interface Layout Geometry

The UI splits the 64-pixel vertical space into three strict windows:

- Top Ribbon (0 to 14px): Displays digital values for BPM and percentage bounds.
- Graph Canvas (15 to 55px): Displays the continuous filtered AC waveform.
- Bottom Bar (56 to 64px): Visual alignment limit line.



2. Diagnostic & Use Checklist

1. **Zero-Power Auto-Reset:** Verify that when a finger is completely removed, the raw IR signal falls below 25,000. The interface must instantly transition from displaying metric values to rendering a centered "PLACE FINGER / ON THE LENS" layout using the optimized 5-pixel width typography matrix.
2. **Contact Pressure:** Place the finger pad lightly over the sensor. Excess force will compress the arterial capillary bed, which zero-flattens the AC pulse and prevents the array from registering local midpoint inflections.

III. Play & Share

- **Implement Signal Quality Index (SQI) Detection:** Modify the block analysis pipeline to compute the variance or standard deviation of peak heights across a window. If the signal is too erratic due to finger movement, display a warning banner like "MOTION ARTIFACT" instead of updating the BPM.
- **Perfusion Index (PI) Calculation:** Calculate the Perfusion Index ($\text{PI} = (\text{AC}/\text{DC}) \times 100\%$). Render a dynamic strength meter on the top OLED bar to guide users on whether their finger pressure is optimal.
- **Pulse Audio Beeper / Haptic Metronome:** Add a piezo buzzer connected to a free GPIO pin. Program the system to sound a short tone on each detected arterial peak, replicating clinical patient monitoring systems.

Use this section in case of parts variation

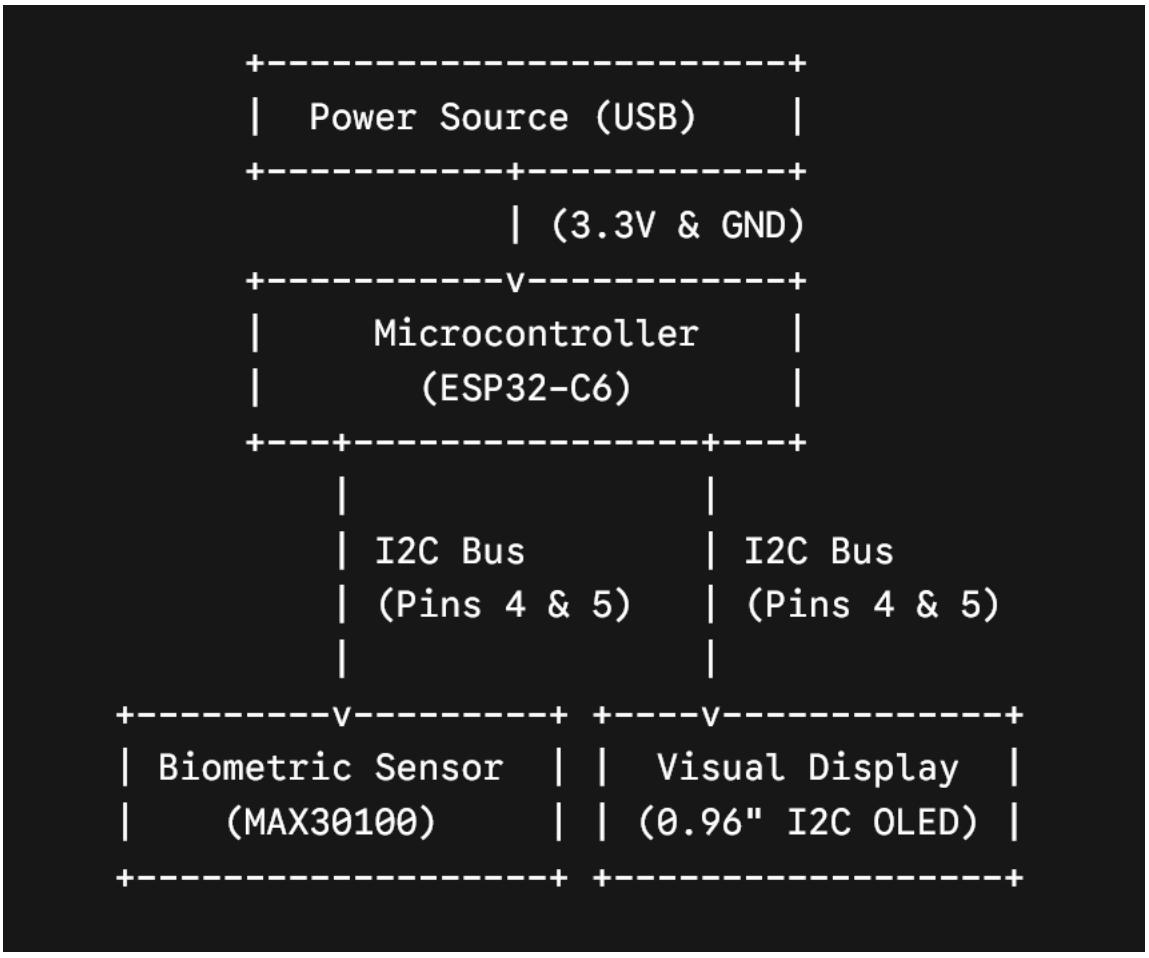
Hello! This section is designed to assist in the case of components varience. In simpler terms, did you get a component that has the same functions but the connections are different?

Some examples include:

- Your OLED Display has 4 pins instead of 7 pins? It turns out you got an I2C Version!
- Your buzzer module is either 2 pins or 3 pins
- Your sensors are a different variation

For the Smart Pulse Oximeter, we are now focusing on how to use the I2C OLED display and an older MAX30100 heart rate sensor. The wiring should be noticeably simplier to work with.

Block Diagram:



You will need to download the "**MAX30100lib**" library in your Arduino IDE's Library Manager

Connections (I2C Version)

Component Pin	ESP32-C6 Pin	Description
Sensor GND & **OLED GND**	GND	Common ground connection (Minus).
Sensor VIN & **OLED VCC**	3.3V	Provides logic power to turn the modules on.
Sensor SDA & **OLED SDA**	GPIO 4	The shared Data line for communication.
Sensor SCL & **OLED SCL**	GPIO 5	The shared Clock line to keep communication timed correctly.

Code

```
#include <Arduino.h>
#include <Wire.h>
#include <U8g2lib.h>
#include <MAX30100.h> // Updated library for the older MAX30100

// --- Shared I2C Pin Configuration ---
#define I2C_SDA 4
#define I2C_SCL 5

// --- Initialize the I2C OLED Display ---
U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE);
MAX30100 particleSensor;

const int SCREEN_WIDTH = 128;
int waveBuffer[SCREEN_WIDTH];

const int BUFFER_SIZE = 100;
uint16_t redBuffer[BUFFER_SIZE];
uint16_t irBuffer[BUFFER_SIZE];
int bufferIndex = 0;

float dcIR = 0;
const float beta = 0.95f; // Filter strength for the heartbeat wave

float finalBpm = 0.0f;
float finalSpO2 = 0.0f;
bool fingerPresent = false;

void setup() {
  Serial.begin(115200);

  // Start the I2C bus on our chosen pins
  Wire.begin(I2C_SDA, I2C_SCL);
  oled.begin();

```

```
  if (!particleSensor.begin()) {
    Serial.println("MAX30100 not found. Check wiring or the 1.8V pull-up issue!");
    while (1); // Stop the program if sensor is missing
  }

  // Configure the MAX30100 for SpO2 and HR mode at 100Hz
  particleSensor.setMode(MAX30100_MODE_SPO2_HR);
  particleSensor.setLedsCurrent(MAX30100_LED_CURR_50MA, MAX30100_LED_CURR_50MA);
  particleSensor.setSamplingRate(MAX30100_SAMPRATE_100HZ);

  // Fill the screen buffer with a flat line to start
  for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32;
}

void loop() {
  particleSensor.update(); // Tell the library to fetch new data

  uint16_t irSample, redSample;

  // If there is a new valid reading from the sensor...
  if (particleSensor.getRawValues(&irSample, &redSample)) {

    // 1. Check if a finger is actually on the sensor
    if (irSample < 7000) { // MAX30100 scales slightly differently than the 30102
      fingerPresent = false;
      finalBpm = 0.0f;
      finalSpO2 = 0.0f;
      bufferIndex = 0;
      for (int i = 0; i < SCREEN_WIDTH; i++) waveBuffer[i] = 32; // Reset wave to flat
    } else {
      fingerPresent = true;

      // 2. Filter the signal to draw the heartbeat wave on the screen
      dcIR = (beta * dcIR) + ((1.0f - beta) * irSample);
      float acDisplay = irSample - dcIR;

      // Shift the old drawing left by 1 pixel to make it scroll
      for (int i = 0; i < SCREEN_WIDTH - 1; i++) {
        waveBuffer[i] = waveBuffer[i + 1];
      }

      // Add the newest pulse reading to the right edge of the screen
      int yPixel = 42 - (int)(acDisplay * 0.05f);
      waveBuffer[SCREEN_WIDTH - 1] = constrain(yPixel, 18, 54);

      // 3. Save the data into our batch array
      redBuffer[bufferIndex] = redSample;
      irBuffer[bufferIndex] = irSample;
      bufferIndex++;

      // 4. Once we have 100 readings (1 second of data), do the math!
      if (bufferIndex >= BUFFER_SIZE) {
        uint16_t minRed = redBuffer[0], maxRed = redBuffer[0];
        uint16_t minIR  = irBuffer[0],  maxIR  = irBuffer[0];
        uint64_t sumRed = 0, sumIR = 0;

        for (int i = 0; i < BUFFER_SIZE; i++) {
          if (redBuffer[i] < minRed) minRed = redBuffer[i];
          if (redBuffer[i] > maxRed) maxRed = redBuffer[i];
          if (irBuffer[i]  < minIR)  minIR  = irBuffer[i];
          if (irBuffer[i]  > maxIR)  maxIR  = irBuffer[i];
          sumRed += redBuffer[i];
          sumIR  += irBuffer[i];
        }

        float dcR_calc = (float)sumRed / BUFFER_SIZE;
        float dcI_calc = (float)sumIR / BUFFER_SIZE;
        float acR_calc = (float)(maxRed - minRed);
        float acI_calc = (float)(maxIR - minIR);

        // 5. Calculate Blood Oxygen (SpO2)
        if (acI_calc > 10 && acR_calc > 10) {
```

```
          float R = (acR_calc / dcR_calc) / (acI_calc / dcI_calc);
          float spo2Est = 104.0f - (17.0f * R);
          if (spo2Est >= 90.0f && spo2Est <= 100.0f) {
            finalSpO2 = spo2Est;
          }
        }

        // 6. Calculate Heart Rate (BPM) by counting the peaks
        int lastPeakIdx = -1;
        int totalIntervals = 0;
        int intervalSum = 0;
        float localThresh = dcI_calc;

        for (int i = 1; i < BUFFER_SIZE - 1; i++) {
          if (irBuffer[i] > localThresh && irBuffer[i-1] <= localThresh) {
            if (lastPeakIdx != -1) {
              intervalSum += (i - lastPeakIdx);
              totalIntervals++;
            }
            lastPeakIdx = i;
          }
        }

        if (totalIntervals > 0) {
          float avgIntervalSamples = (float)intervalSum / totalIntervals;
          float calculatedBpm = (60.0f * 100.0f) / avgIntervalSamples;
          if (calculatedBpm >= 50.0f && calculatedBpm <= 160.0f) {
            finalBpm = calculatedBpm;
          }
        }
        bufferIndex = 0; // Reset the batch counter for the next second
      }
    }
  }

  // 7. Draw the graphics to the OLED Screen
  oled.clearBuffer();

  if (fingerPresent && finalBpm > 10.0f) {
    for (int x = 1; x < SCREEN_WIDTH; x++) {
      oled.drawLine(x - 1, waveBuffer[x - 1], x, waveBuffer[x]);
    }
    oled.setFont(u8g2_font_6x10_tf);
    oled.setCursor(2, 11);  oled.print("BPM: "); oled.print((int)finalBpm);
    oled.setCursor(72, 11); oled.print("SpO2: "); oled.print((int)finalSpO2); oled.print("%");
  } else {
    oled.setFont(u8g2_font_5x7_tf);
    oled.setCursor(34, 32); oled.print("PLACE FINGER");
    oled.setCursor(36, 44); oled.print("ON THE LENS");
  }

  oled.drawHLine(0, 14, 128);
  oled.drawHLine(0, 56, 128);
  oled.sendBuffer();
}

```